FigForge 科研可视化与图件生产平台需求文档（完整版）

一、项目背景

在当前生物信息学与医学科研实践中，分析计算高度依赖 R/Python，而图件制作仍大量依赖 Photoshop、Illustrator 或 PowerPoint 进行手工拼版。该流程存在效率低、不可复现、返工成本高、图件风格不统一等系统性问题。

FigForge 旨在构建一套面向科研的图件生产平台，通过工程化方式重构“分析 → 作图 → 排版 → 投稿导出”的完整流程。

二、产品定位与目标

产品定位：面向生物信息学与医学科研的 Figure Factory（图件生产平台）。

核心目标：

1）将生信分析与图件排版解耦；

2）以本地 R 计算为起点，控制早期成本；

3）通过前端交互式排版替代 Photoshop；

4）支持未来无痛迁移至服务器计算；

5）最终实现图件输出的可复现、可规范、可规模化。

三、总体设计原则

1）分层架构：计算层、渲染层、排版层、导出层相互解耦；

2）本地优先：默认在用户本机运行分析；

3）协议驱动：统一 Job 输入/输出规范；

4）渐进演进：允许逐步替换渲染引擎；

5）科研友好：严格对齐 SCI 期刊图件规范。

四、系统架构概述

系统由四个核心模块组成：

1）计算引擎（Analysis Engine）：以 R 为核心的分析 pipeline；

2）渲染引擎（Rendering Engine）：用于生成单 panel 图件；

3）排版编辑器（Layout Editor）：前端交互式拼版与标注；

4）导出引擎（Export Engine）：生成投稿级文件。

各模块通过标准化数据协议连接，可独立演进。

五、计算引擎需求（Analysis Engine）

1）支持调用用户本机 R / Rscript；

2）支持多 pipeline（单细胞、bulk、显微图等）；

3）支持 pipeline 版本化（pipeline_id）；

4）支持 JSON 参数输入；

5）支持进度回传与日志输出；

6）统一输出目录结构：

- result.json
- panels/*.svg
- tables/*.csv
- artifacts.json

7）保证同输入同参数结果结构一致。

六、渲染引擎需求（Rendering Engine）

采用分阶段策略：

阶段一：R/ggplot 输出 SVG

- 确保分析结果的可信性与科研接受度；

阶段二：前端渲染（可选）

– 对高频图形（UMAP、volcano、QC）支持前端重绘；

– 输入为标准化中间数据（JSON）；

阶段三：多渲染器并存

– ggplot / Plotly / ECharts / Vega 等并行支持；

– 排版层对渲染器无感知。

七、排版编辑器需求（Layout Editor）

1）支持 SVG/PNG/TIFF 等多种 panel 类型；

2）支持多 panel 画布（A/B/C…）；

3）支持拖拽、缩放、裁剪、旋转；

4）支持自动对齐、吸附、等间距；

5）支持自动 panel 标号；

6）支持文本、箭头、比例尺等标注；

7）支持全局字体、字号、线宽控制；

8）支持保存与恢复排版状态（layout.json）；

9）所有排版操作不触发重新计算。

八、导出引擎需求（Export Engine）

1）支持 SVG / PDF 矢量导出；

2）支持 PNG / TIFF 位图导出；

3）支持 300 dpi / 600 dpi；

4）支持期刊模板（Nature 单/双栏）；

5）支持字体嵌入与颜色规范；

6）支持一键导出投稿版本。

九、性能与用户体验要求

1）分析计算与 UI 渲染分离；

2）支持任务队列、取消、失败恢复；

3）支持大文件与高分辨率图像；

4）支持实时进度与日志查看；

5）界面交互需保持毫秒级响应。

十、产品演进路线

阶段 A：本地单机版（当前）

– 桌面端 + 本地 R + 排版器

阶段 B：混合模式

– 本地计算 + 服务器计算可选

阶段 C：云端产品

– 容器化 pipeline

– 多用户、权限、配额、审计

始终保留本地模式以满足隐私与合规需求。

十一、目标用户与应用场景

目标用户：

– 生物信息学研究人员

– 临床科研医生

– 研究生与实验室技术人员

应用场景：

– SCI 论文图件制作

– 基金与课题汇报

– 多组学结果可视化

– 显微/WB/IF 图像拼版

十二、价值总结

FigForge 通过引入工程化、产品化思维，系统性解决科研图件生产中的效率与复现问题。

它不是简单的画图工具，而是科研生产流程中的关键基础设施。